

X


<https://swayam.gov.in>

https://swayam.gov.in/nc_details/NPTEL

riya.soni17oct@gmail.com ✓

NPTEL (<https://swayam.gov.in/explorer?ncCode=NPTEL>) » Design and Analysis of Algorithms (course)

If already
registered, click
to check your
payment status

Course
outlineAbout
NPTEL ()How does an
NPTEL
online
course
work? ()Week 1 :
Introduction
()Week 1 :
Analysis of
algorithms ()Week 1 Quiz
()Week 2 :
Searching
and sorting
()

Week 5 Quiz

The due date for submitting this assignment has passed.

Due on 2025-08-27, 23:59 IST.

Assignment submitted on 2025-08-27, 15:47 IST

All questions carry equal weightage. You may submit as many times as you like within the deadline. Your final submission will be graded.

1) Suppose we want to extend the union-find data structure to support the operation **Reset(c)**, which takes as input the name of a component c and then breaks up c into singleton components, like **MakeUnionFind()**. For instance if $c = 3$ and c currently consists of $\{1, 3, 7\}$, then **Reset(c)** will produce three components called 1, 3 and 7 consisting of $\{1\}$, $\{3\}$ and $\{7\}$, respectively. **1 point**

Which of the following is correct about the cost of adding **Reset(c)** to the array and pointer implementations of union-find discussed in the lecture?

- ☒ Array representation: $O(n)$, Pointer representation: $O(\text{size}(c))$
- ☐ Array representation: $O(\text{size}(c))$, Pointer representation: $O(\text{size}(c))$
- ☐ Array representation: $O(\text{size}(c))$, Pointer representation: $O(n)$
- ☐ Array representation: $O(n)$, Pointer representation: $O(n)$

No, the answer is incorrect.

Score: 0

Feedback:

In the array representation we have the list $\text{Members}[c]$ which allows us to update the contents of c in time $O(\text{size}(c))$. In the pointer representation there is no easy way to identify all elements that belong to component c without scanning the entire set, so it takes time $O(n)$

Accepted Answers:

Array representation: $O(\text{size}(c))$, Pointer representation: $O(n)$

Week 2 Quiz ()

Week 2 Programmin g Assignment ()

Week 3 : Graphs ()

Week 3 Quiz ()

Week 3 Programmin g Assignment ()

Week 4 : Weighted graphs ()

Week 4 Quiz ()

Week 4 Programmin g Assignment ()

Week 5: Data Structures: Union-Find and Heaps ()

Week 5 : Divide and Conquer ()

Week 5 Quiz ()

Quiz: Week 5 Quiz (assessment? name=241)

2) In the algorithm presented for the closest pair of points problem, we have assumed **1 point** that no two points have the same x or y coordinate. Which of the following steps would become more complicated to justify without this assumption.

- ☒ Arguing that every $d/2$ side square in the d-band around the separator can have at most one point.
- ☐ Constructing S_Y from Q_Y and R_Y in time $O(n)$ in the combine step.
- ☐ Constructing Q_X and R_X from P_X in time $O(n)$ in the divide step.
- ☐ Constructing Q_Y and R_Y from P_Y in time $O(n)$ in the divide step.

No, the answer is incorrect.

Score: 0

Feedback:

All x-coordinates could be the same, in which case we may need $O(n)$ time for each point in P_Y to assign it to Q_Y or R_Y .

Accepted Answers:

Constructing Q_Y and R_Y from P_Y in time $O(n)$ in the divide step.

3) Suppose we want to support the operations predecessor and successor in a heap. **1 point** Given a value v in the heap, $\text{pred}(v)$ tells us the next smaller value currently in the heap and $\text{succ}(v)$ tells us the next larger value currently in the heap.

- ☒ In both min heaps and max heaps, both operations take time $O(n)$.
- ☐ In both min heaps and max heaps, both operations take time $O(\log n)$.
- ☐ In a min heap, $\text{pred}(v)$ takes time $O(\log n)$ and $\text{succ}(v)$ takes $O(n)$ whereas in a max heap $\text{pred}(v)$ takes time $O(n)$ and $\text{succ}(v)$ takes $O(\log n)$.
- ☐ In a min heap, $\text{pred}(v)$ takes time $O(n)$ and $\text{succ}(v)$ takes $O(\log n)$ whereas in a max heap $\text{pred}(v)$ takes time $O(\log n)$ and $\text{succ}(v)$ takes $O(n)$.

Yes, the answer is correct.

Score: 1

Feedback:

In either case, there is no obvious way to compute $\text{pred}(v)$ and $\text{succ}(v)$ without scanning all elements, so both cases take $O(n)$.

Accepted Answers:

In both min heaps and max heaps, both operations take time $O(n)$.

4) Consider the min-heap [12, 27, 48, 30, 37, 79, 54, 43, 39] built by repeatedly **1 point** inserting values into an empty heap. Which of the following *could not* have been the last element inserted into this heap?

- ☐ 27
- ☐ 30
- ☒ 37
- ☐ 39

Yes, the answer is correct.

Score: 1

Feedback:

The last position added was the one containing 39. The last element added must lie on the path from 39 to the root: {12, 27, 30, 39}.

Accepted Answers:

37

Week 6: Data Structures: Search Trees
()

Week 6: Greedy Algorithms
()

Week 6 Quiz
()

Week 6 Programming Assignment
()

Week 7: Dynamic Programming
()

Week 7 Quiz
()

Week 7 Programming Assignment
()

Download Videos
()

Lecture Material
()

Problem Solving Session - July 2025
()

5) Suppose we implement merge sort with a five-way split: divide the array into 5 equal parts, sort each part and do a 5 way merge. What would the worst-case complexity of this version be? **1 point**

- ☐ $O(n^2)$
- ☐ $O(n^2 \log_5 n)$
- ☒ $O(n \log_2 n)$
- ☐ $O(n (\log_2 n)^2)$

Yes, the answer is correct.

Score: 1

Feedback:

The recurrence will yield $O(n \log_5 n)$, but $\log_5 n = \log_2 n / \log_2 5 = O(\log_2 n)$.

Accepted Answers:

$O(n \log_2 n)$

